# LLARS v2 - AI Script Authoring Reference

A source-grounded implementation reference for building compatible LLARS v2 AutoHotkey v1 scripts.

Purpose: produce safe, normal LLARS scripts that use the actual public Creator API. This is a task-oriented implementation reference, not a replacement for the installed LLARS source or Windows testing.

Authoritative sources in the project: Core/LLARS_CreatorCatalog.ahk (API), Core/LLARS_CreatorAPI.ahk (implementation), Core/LLARS_Config.ahk (config), Templates/RunCount/Basic Loop (scaffold), Core/LLARS_CreatorRegistry.ahk (generated-selector registration), and representative scripts in both script-family folders.

## 1. Script construction contract

Output a normal .ahk activity script, its Config.ini when required, and a concise list of Setup requirements. Preserve LLARS bootstrap, include logic, run-state behavior, and Script Selector format. Never invent unknown framework functions or private config keys.

Input checklist: script purpose; Normal or Menu Entry Swapper family; RunCount or Timer template; target coordinate/colour/hotkey names; action sequence; timing ranges; required versus optional config; expected stop/pause/focus behavior; whether Selector registration is wanted.

| DO | DO NOT |
|---|---|
| Edit a copy of the matching current template. | Write a second script engine or replace Core runtime. |
| Use only catalogued LLARS_* calls with valid signatures. | Call internal-only helpers simply because their names look useful. |
| Use LLARS_Click, LLARS_PressHotkey, LLARS_Sleep and the corresponding APIs. | Use raw Send, Click, MouseClick, MouseMove, direct AHI or Win32 input injection. |
| Define every used typed Config.ini section. | Guess live client coordinates or fill unknown game input values. |
| Guard runtime actions and retain run-state/pause safety. | Start live automation during a generation or static test. |
| Preserve originals, use unique script names, test on Windows. | Silently overwrite an existing script or claim field PASS from source review. |

## 2. Deterministic authoring procedure

1. Inspect the installed LLARS v2 project and confirm it uses a compatible AutoHotkey v1 interpreter.
2. Choose script family: Scripts for Normal, Scripts - Menu Entry Swapper for MES.
3. Select a current template matching the lifecycle (RunCount, Timer, MultiTimer, Color or MultiColor).
4. Copy the complete scaffold, change only the activity logic, and retain framework/bootstrap includes.
5. Map every proposed action to an exact Creator catalog entry and its parameter types.
6. Create one typed Config.ini section per distinct dependency; reuse shared names and use valid typed keys.
7. Place user-visible status/detection and at least one enabled configured wait in a RunCount cycle.
8. Use modeled runtime metadata only when the timing path is knowable; otherwise rely on parser fallback.

9. Validate AHK structure, config references, capability requirements, file paths and uniqueness.

10. Review with a person and conduct Windows Setup/input/Selector/RunCount tests before runtime approval.

## 3. File placement and script lifecycle

```
LLARS v2/
  Core/LLARS_ScriptBootstrap.ahk
  Scripts/<Script Name>/<Script Name>.ahk
  Scripts/<Script Name>/Config.ini
  Scripts - Menu Entry Swapper/<Script Name>/<Script Name>.ahk
  Scripts - Menu Entry Swapper/<Script Name>/Config.ini
```

Both script families consume the same framework. Normal banking uses the configured Bank Preset action; MES banking omits that action and its associated config. Do not turn an MES activity into a Normal activity merely by changing its directory.

For RunCount, prefer Templates/RunCount/Basic Loop/Basic Loop Script Template.ahk. Retain #Requires, LLARS_SCRIPT_TYPE, LLARS_FrameworkAvailable, LLARS_Initialize, the Start label, LLARS_RunCount(Func("Run")), editable markers, return statements and existing #Include search paths. If a script lives deeper in the directory tree, keep all tested include-fallback lines.

```
Run(ctx)
{
    global
    LLARS_SetStatus("Working", "Action")
    LLARS_Click("Action Target")
    LLARS_Sleep("Action Delay", ctx.IsLast)
}
```

For Timer or MultiTimer choose their existing templates. Do not copy RunCount-specific startup into a timer script. Managed timer callback examples exist in the Script Creator Library; use LLARS_TimerOnce/LLARS_TimerRepeat and stop handles as appropriate.

## 4. Config.ini schemas - exact Creator catalog layouts

| Type | Example section (script-local Config.ini) | Notes |
|---|---|---|
| coordinate / Point | [Action Target] type=coordinate x= y= | Fixed client-coordinate point; leave unknown values for Setup. |
| coordinate / Area | [Bank] type=coordinate xmin= xmax= ymin= ymax= | Randomized rectangle; no duplicate section names. |
| hotkey | [Action Hotkey] type=hotkey hotkey= | Set through Setup or deliberate author defaults. |
| color | [Target Color] type=color color=0xFFFFFF | RGB hex 0xRRGGBB; verify live color. |
| timing_range | [Action Delay] min=500 max=1000 | ms; min <= max; optional type omitted. |
| timer | [Script Timer] type=timer option=false min=10000 max=30000 | Enabled/disabled recurring timer. |
| option | [Optional Feature] option=true | Optional section switch. Other keys must be specified deliberately. |

Text above lists schema keys compactly for lookup; write a real INI as one key per line, not a single-line section. Use an encoding compatible with the installed LLARS Config reader and Windows INI functions. Scope defaults to "script"; use shared root scope only for truly shared LLARS settings.

```
[Action Target]
type=coordinate
x=
y=

[Action Hotkey]
type=hotkey
hotkey=

[Action Delay]
min=500
max=1000
```

All literal config references in the generated AHK must resolve. Sections used by multiple actions are shared. Removing an action does not authorize deleting a retained section without consent. Do not require Config.ini when no local section is needed.

## 5. Code recipe: one repeating action

Input: click a configured coordinate and wait. Choose Basic Loop; do not generate a separate Loop around Run(ctx): RunCount owns iteration.

```
Run(ctx)
{
    global
    LLARS_SetStatus("Working", "Target")
    LLARS_Click("Action Target")
    LLARS_Sleep("Action Delay", ctx.IsLast)
}
```

```
[Action Target]
type=coordinate
x=
y=

[Action Delay]
min=750
max=1250
```

Treat x/y as user Setup inputs; do not invent values. LLARS_Sleep contributes to RunCount estimate and preserves shared pause/run behavior.

## 6. Code recipe: detected color with branch

```
Run(ctx)
{
    global
    if LLARS_PixelMatches("Target Pixel", "Target Color")
    {
        LLARS_SetDetection("Target", "Matched")
        LLARS_Click("Action Target")
    }
    else
        LLARS_SetDetection("Target", "Not found")
    LLARS_Sleep("Scan Delay", ctx.IsLast)
}
```

```
[Target Pixel]
type=coordinate
x=
y=

[Target Color]
type=color
color=0xFFFFFF

[Action Target]
type=coordinate
x=
y=

[Scan Delay]
min=500
max=1000
```

0xFFFFFF is a schema example, not a validated target color. If color capture or detection is unverified, request a real value instead of fabricating gameplay assumptions.

## 7. Code recipe: Normal vs MES banking

### Normal preset-based sequence

```
LLARS_SetStatus("Banking", "Open bank")
LLARS_Click("Bank")
LLARS_Sleep("Bank Wait")
LLARS_SetStatus("Loading preset", "Normal banking")
LLARS_PressHotkey("Bank Preset")
LLARS_Sleep("Preset Wait")
```

### Menu Entry Swapper sequence

```
LLARS_SetStatus("Banking", "Open bank")
LLARS_Click("Bank")
LLARS_Sleep("Bank Wait")
; Continue to the next action. No Bank Preset hotkey.
```

For MES, generate under Scripts - Menu Entry Swapper and omit the Bank Preset section entirely. For Normal, generate under Scripts and define [Bank Preset] type=hotkey. Do not include "Loading preset" status in MES. Preserve other production inputs and timing if converting between modes. Do not rewrite user-authored arbitrary AHK.

## 8. RunCount timing and estimation

Use named LLARS_Sleep sections. Optional LLARS_RandomSleep draws from shared LLARS Config.ini. RunCount metadata is an optimization only when all timing paths are known. For custom code, unknown conditional branch probabilities, Repeat Forever, Break/Continue, or unsupported timing, omit synthesized metadata and let the existing source parser estimate what it can.

```
LLARS_RuntimeEstimateMetadata()
{
    model := LLARS_RuntimeMetadataCreate("modeled")
    LLARS_RuntimeMetadataAddSection(model, "Action Delay", 2)
    LLARS_RuntimeMetadataAddRandomSleep(model)
    return model
}
```

Do not hard-code expected total duration; it depends on current config and optional random sleeps. Only add metadata functions to a compatible RunCount script and ensure all referenced sections exist. Infinite-loop or uncertain custom paths must not claim precise modeled confidence.

## 9. Selector and Setup integration

Use the built-in script-creation workflow when Selector registration is wanted. The registered entries must coexist with shipped entries; never modify existing authored Selector entries.

Manual script generation produces an independently runnable .ahk plus required Config.ini. Selector registration is optional; use the supported registry interface rather than inventing registry keys. Setup requirements must be derived from configuration with the shared scanner, not separately declared Requires* flags.

Creation must be collision-safe. If a destination folder already exists, choose another name or ask the user. Register only after successful post-write validation. Test Script means real automation, not a harmless dry-run.

When using a visual editor, retain its optional project data if you want to reopen an editable action sequence. Generated .ahk and Config.ini files must remain independently usable. Never assume arbitrary hand-written or externally modified AHK can be imported and rewritten safely; inspect and edit it as source code.

## 10. Verification and handoff contract

| Check | Required evidence |
|---|---|
| Source and scaffold | AHK v1 syntax, editable markers, includes, LLARS_Initialize, Start/Run signatures. |
| API usage | Every LLARS_* call exists in catalog or a separately verified framework contract; correct arity and types. |
| Config | Every literal section resolves; types/keys/defaults valid; no duplicate sections. |
| Capability parity | Generated config has the same requirements as the shared scanner and Setup. |
| Normal/MES | Correct family path, preset semantics, no contradictory MES preset metadata. |
| Input and focus | No raw Send/Click or direct AHI; pause/stop/focus logic preserved. |
| RunCount | At least one enabled safe wait, model only known timing; no invented run-duration guarantees. |
| Files and selector | No silent overwrite; optional registry entry validates and persists. |
| Windows acceptance | GUI opens, Setup configures, Selector loads, generated script starts/stops safely. |

Delivery checklist: (1) exact file paths, (2) complete AHK source, (3) complete Config.ini or explicit absence, (4) Setup values user must supply, (5) family/Selector registration plan, (6) validation findings and any field-test limitations. Do not state Windows functionality is verified unless actually exercised.

# 11. API function reference

Use the signatures, parameter types and examples below when writing AHK. Consult the installed LLARS source for authoritative availability and behavior; compatibility functions should only be used when necessary for existing code.

## Creator Public

### LLARS_FindColorCenterOut Colors

```
LLARS_FindColorCenterOut(colorSections, ByRef foundX, ByRef foundY, ByRef matchedSection, avoidSection := "", searchName := "Color", avoidX := "", avoidY := "", scope := "script", tolerance := 0, allowAvoidFallback := false, requireNeighborhood := false)
```

Params: colorSections: config_color_list [required]; foundX: out_number [required] [ByRef]; foundY: out_number [required] [ByRef]; matchedSection: out_string [required] [ByRef]; avoidSection: config_color_or_blank = ""; searchName: string = "Color"; avoidX: number_or_blank = ""; avoidY: number_or_blank = ""; scope: scope = "script"; tolerance: integer = 0; allowAvoidFallback: boolean = false; requireNeighborhood: boolean = false

Config: color | Return: boolean | Input/block: no/no

Example: `if LLARS_FindColorCenterOut(["Target Color"], foundX, foundY, matchedSection)`

### LLARS_VerifyColorTarget Colors

```
LLARS_VerifyColorTarget(ByRef x, ByRef y, colorSection, actionName := "Color", scope := "script", radius := 28, tolerance := 0, confirmationDelay := 0, confirmationRadius := "")
```

Params: x: out_number [required] [ByRef]; y: out_number [required] [ByRef]; colorSection: config_color [required]; actionName: string = "Color"; scope: scope = "script"; radius: integer = 28; tolerance: integer = 0; confirmationDelay: integer = 0; confirmationRadius: integer_or_blank = ""

Config: color | Return: boolean | Input/block: no/no

Example: `if LLARS_VerifyColorTarget(foundX, foundY, matchedSection)`

### LLARS_PixelChangedFrom Conditions

```
LLARS_PixelChangedFrom(coordinateSection, baselineColor, ByRef currentColor, scope := "script")
```

Params: coordinateSection: config_coordinate [required]; baselineColor: color_hex [required]; currentColor: out_color_hex [required] [ByRef]; scope: scope = "script"

Config: coordinate | Return: boolean | Input/block: no/no

Example: `if LLARS_PixelChangedFrom("Pixel Coordinate", baselineColor, currentColor)`

### LLARS_PixelColor Conditions

```
LLARS_PixelColor(coordinateSection, scope := "script")
```

Params: coordinateSection: config_coordinate [required]; scope: scope = "script"

Config: coordinate | Return: color_hex_or_blank | Input/block: no/no

Example: `currentColor := LLARS_PixelColor("Pixel Coordinate")`

### LLARS_PixelMatches Conditions

```
LLARS_PixelMatches(coordinateSection, colorSection, scope := "script")
```

Params: coordinateSection: config_coordinate [required]; colorSection: config_color [required]; scope: scope = "script"

Config: coordinate,color | Return: boolean | Input/block: no/no

Example: `if LLARS_PixelMatches("Pixel Coordinate", "Target Color")`

### LLARS_PixelMatchesAny Conditions

```
LLARS_PixelMatchesAny(coordinateSection, colorSections, scope := "script")
```

Params: coordinateSection: config_coordinate [required]; colorSections: config_color_list [required]; scope: scope = "script"

Config: coordinate,color | Return: section_name_or_blank | Input/block: no/no

Example: `matched := LLARS_PixelMatchesAny("Pixel Coordinate", ["State One", "State Two"])`

### LLARS_PixelMatchesWithinTolerance Conditions

`LLARS_PixelMatchesWithinTolerance(coordinateSection, colorSection, tolerance := 0, scope := "script")`

Params: coordinateSection: config_coordinate [required]; colorSection: config_color [required]; tolerance: integer = 0; scope: scope = "script"

Config: coordinate,color | Return: boolean | Input/block: no/no

Example: `if LLARS_PixelMatchesWithinTolerance("Pixel Coordinate", "Target Color", 3)`

### LLARS_ConfigEnabled Configuration

`LLARS_ConfigEnabled(section, default := true, scope := "script")`

Params: section: config_section [required]; default: any = true; scope: scope = "script"

Config: config_section | Return: boolean | Input/block: no/no

Example: `if LLARS_ConfigEnabled("Optional Feature")`

### LLARS_ConfigRead Configuration

`LLARS_ConfigRead(section, key, default := "ERROR", scope := "script")`

Params: section: config_section [required]; key: config_key [required]; default: any = "ERROR"; scope: scope = "script"

Config: config_section | Return: any_or_default | Input/block: no/no

Example: `value := LLARS_ConfigRead("Options", "value", "")`

### LLARS_ConfigReadBool Configuration

`LLARS_ConfigReadBool(section, key := "option", default := false, scope := "script")`

Params: section: config_section [required]; key: config_key = "option"; default: boolean = false; scope: scope = "script"

Config: config_section | Return: boolean | Input/block: no/no

Example: `enabled := LLARS_ConfigReadBool("Feature")`

### LLARS_ConfigReadChoice Configuration

`LLARS_ConfigReadChoice(section, key, choices, default := "", required := false, scope := "script")`

Params: section: config_section [required]; key: config_key [required]; choices: string_list [required]; default: any = ""; required: boolean = false; scope: scope = "script"

Config: config_section | Return: string_or_default | Input/block: no/no

Example: `mode := LLARS_ConfigReadChoice("Options", "mode", ["Fast", "Normal"], "Normal")`

### LLARS_ConfigReadColor Configuration

`LLARS_ConfigReadColor(section, key := "", scope := "script")`

Params: section: config_color [required]; key: config_key = ""; scope: scope = "script"

Config: color | Return: color_hex_or_blank | Input/block: no/no

Example: `color := LLARS_ConfigReadColor("Target Color")`

### LLARS_ConfigReadHotkey Configuration

`LLARS_ConfigReadHotkey(section, key := "hotkey", scope := "script")`

Params: section: config_hotkey [required]; key: config_key = "hotkey"; scope: scope = "script"

Config: hotkey | Return: hotkey_or_blank | Input/block: no/no

Example: `hotkey := LLARS_ConfigReadHotkey("Action Hotkey")`

### LLARS_ConfigReadInteger Configuration

`LLARS_ConfigReadInteger(section, key, default := "", minAllowed := "", maxAllowed := "", scope := "script")`

Params: section: config_section [required]; key: config_key [required]; default: any = ""; minAllowed: number_or_blank = ""; maxAllowed: number_or_blank = ""; scope: scope = "script"

Config: config_section | Return: integer_or_default | Input/block: no/no

Example: `count := LLARS_ConfigReadInteger("Options", "count", 1, 1, 100)`

### LLARS_ConfigReadNumber Configuration

`LLARS_ConfigReadNumber(section, key, default := "", minAllowed := "", maxAllowed := "", scope := "script")`

Params: section: config_section [required]; key: config_key [required]; default: any = ""; minAllowed: number_or_blank = ""; maxAllowed: number_or_blank = ""; scope: scope = "script"

Config: config_section | Return: number_or_default | Input/block: no/no

Example: `threshold := LLARS_ConfigReadNumber("Options", "threshold", 0.5, 0, 1)`

### LLARS_ConfigReadPoint Configuration

`LLARS_ConfigReadPoint(section, ByRef x, ByRef y, scope := "script")`

Params: section: config_coordinate [required]; x: out_number [required] [ByRef]; y: out_number [required] [ByRef]; scope: scope = "script"

Config: coordinate | Return: boolean | Input/block: no/no

Example: `LLARS_ConfigReadPoint("Action Location", x, y)`

### LLARS_ConfigReadRange Configuration

`LLARS_ConfigReadRange(section, ByRef minValue, ByRef maxValue, scope := "script")`

Params: section: config_timing_range [required]; minValue: out_number [required] [ByRef]; maxValue: out_number [required] [ByRef]; scope: scope = "script"

Config: timing_range | Return: boolean | Input/block: no/no

Example: `LLARS_ConfigReadRange("Action Delay", minValue, maxValue)`

### LLARS_ConfigReadString Configuration

`LLARS_ConfigReadString(section, key, default := "", required := false, scope := "script")`

Params: section: config_section [required]; key: config_key [required]; default: any = ""; required: boolean = false; scope: scope = "script"

Config: config_section | Return: string | Input/block: no/no

Example: `name := LLARS_ConfigReadString("Options", "name", "Default")`

### LLARS_ConfigSections Configuration

`LLARS_ConfigSections(scope := "script", default := "ERROR")`

Params: scope: scope = "script"; default: any = "ERROR"

Config: config_section | Return: newline_delimited_string_or_default | Input/block: no/no

Example: `sections := LLARS_ConfigSections()`

### LLARS_ConfigWrite Configuration

`LLARS_ConfigWrite(section, key, value, scope := "script")`

Params: section: config_section [required]; key: config_key [required]; value: any [required]; scope: scope = "script"

Config: config_section | Return: boolean | Input/block: no/no

Example: `LLARS_ConfigWrite("Timer", "min", 5000)`

### LLARS_PressHotkey Keyboard

`LLARS_PressHotkey(section, key := "hotkey", scope := "script")`

Params: section: config_hotkey [required]; key: config_key = "hotkey"; scope: scope = "script"

Config: hotkey | Return: boolean | Input/block: yes/yes

Example: `LLARS_PressHotkey("Action Hotkey")`

### LLARS_PressKey Keyboard

`LLARS_PressKey(keyName)`

Params: keyName: key_name [required]

Config: none | Return: boolean | Input/block: yes/yes

Example: `LLARS_PressKey("Space")`

### LLARS_Click Mouse

`LLARS_Click(section, button := "left", scope := "script")`

Params: section: config_coordinate [required]; button: mouse_button = "left"; scope: scope = "script"

Config: coordinate | Return: boolean | Input/block: yes/yes

Example: `LLARS_Click("Action Location")`

### LLARS_ClickPoint Mouse

`LLARS_ClickPoint(x, y, button := "left", actionName := "Pixel Target")`

Params: x: number [required]; y: number [required]; button: mouse_button = "left"; actionName: string = "Pixel Target"

Config: none | Return: boolean | Input/block: yes/yes

Example: `LLARS_ClickPoint(foundX, foundY, "left", "Detected target")`

### LLARS_RedClick Mouse

`LLARS_RedClick(section, button := "left", scope := "script")`

Params: section: config_coordinate [required]; button: mouse_button = "left"; scope: scope = "script"

Config: coordinate | Return: boolean | Input/block: yes/yes

Example: `LLARS_RedClick("World Object")`

### LLARS_RedClickPoint Mouse

`LLARS_RedClickPoint(x, y, button := "left", actionName := "Verified Pixel Target")`

Params: x: number [required]; y: number [required]; button: mouse_button = "left"; actionName: string = "Verified Pixel Target"

Config: none | Return: boolean | Input/block: yes/yes

Example: `LLARS_RedClickPoint(foundX, foundY, "left", "Detected world object")`

### LLARS_RedClickRetry Mouse

`LLARS_RedClickRetry(section, button := "left", scope := "script")`

Params: section: config_coordinate [required]; button: mouse_button = "left"; scope: scope = "script"

Config: coordinate | Return: boolean | Input/block: yes/yes

Example: `if !LLARS_RedClickRetry("World Object")`

### LLARS_RedClickRetryPoint Mouse

`LLARS_RedClickRetryPoint(x, y, button := "left", actionName := "Retryable Verified Pixel Target")`

Params: x: number [required]; y: number [required]; button: mouse_button = "left"; actionName: string = "Retryable Verified Pixel Target"

Config: none | Return: boolean | Input/block: yes/yes

Example: `if !LLARS_RedClickRetryPoint(foundX, foundY)`

### LLARS_RunActive Script lifecycle

`LLARS_RunActive(requireRuneScape := false)`

Params: requireRuneScape: boolean = false

Config: none | Return: boolean | Input/block: no/no

Example: `if LLARS_RunActive()`

### LLARS_RunCount Script lifecycle

`LLARS_RunCount(callback)`

Params: callback: callback [required]

Config: none | Return: boolean | Input/block: no/yes

Example: `LLARS_RunCount(Func("RunTask"))`

### LLARS_StartTimerRun Script lifecycle

`LLARS_StartTimerRun()`

Params: (no parameters)

Config: none | Return: boolean | Input/block: no/yes

Example: `if !LLARS_StartTimerRun()`

### LLARS_WaitForRuneScape Script lifecycle

`LLARS_WaitForRuneScape(action := "Automation", requireRun := true)`

Params: action: string = "Automation"; requireRun: boolean = true

Config: none | Return: hwnd_or_false | Input/block: no/yes

Example: `if !LLARS_WaitForRuneScape("Target scan")`

### LLARS_Error Status

`LLARS_Error(title, whatHappened, whatLLARSDid := "", whatUserShouldDo := "", detail := "", showMessage := true)`

Params: title: string [required]; whatHappened: string [required]; whatLLARSDid: string = ""; whatUserShouldDo: string = ""; detail: string = ""; showMessage: boolean = true

Config: none | Return: void | Input/block: no/no

Example: `LLARS_Error("Configuration error", "Required setting is invalid.")`

### LLARS_LogEvent Status

`LLARS_LogEvent(summary, detail := "")`

Params: summary: string [required]; detail: string = ""

Config: none | Return: void | Input/block: no/no

Example: `LLARS_LogEvent("Bank cycle", "Opened bank")`

### LLARS_Recovered Status

`LLARS_Recovered(summary, detail := "")`

Params: summary: string [required]; detail: string = ""

Config: none | Return: void | Input/block: no/no

Example: `LLARS_Recovered("Target reacquired", "Nearby scan succeeded")`

### LLARS_SetDetection Status

`LLARS_SetDetection(name, state, confidence := "", threshold := "", detail := "", logChange := true)`

Params: name: string [required]; state: string [required]; confidence: number_or_blank = ""; threshold: number_or_blank = ""; detail: string = ""; logChange: boolean = true

Config: none | Return: void | Input/block: no/no

Example: `LLARS_SetDetection("Target", "Searching")`

### LLARS_SetStatus Status

`LLARS_SetStatus(status, activity := "")`

Params: status: string [required]; activity: string = ""

Config: none | Return: void | Input/block: no/no

Example: `LLARS_SetStatus("Working", "Bank")`

### LLARS_Warn Status

`LLARS_Warn(title, whatHappened, whatLLARSDid := "", whatUserShouldDo := "", detail := "", showMessage := true)`

Params: title: string [required]; whatHappened: string [required]; whatLLARSDid: string = ""; whatUserShouldDo: string = ""; detail: string = ""; showMessage: boolean = true

Config: none | Return: void | Input/block: no/no

Example: `LLARS_Warn("Target not found", "No valid target matched.", "No click was sent.")`

### LLARS_RandomSleep Timing

`LLARS_RandomSleep()`

Params: (no parameters)

Config: shared_random_sleep | Return: milliseconds | Input/block: no/yes

Example: `LLARS_RandomSleep()`

### LLARS_Sleep Timing

`LLARS_Sleep(section, final := false, scope := "script")`

Params: section: config_timing_range [required]; final: boolean = false; scope: scope = "script"

Config: timing_range | Return: milliseconds | Input/block: no/yes

Example: `LLARS_Sleep("Action Delay")`

### LLARS_TimerInterval Timing

`LLARS_TimerInterval(section, scope := "script")`

Params: section: config_timer [required]; scope: scope = "script"

Config: timer | Return: milliseconds | Input/block: no/no

Example: `interval := LLARS_TimerInterval("Script Timer")`

### LLARS_TimerOnce Timing

`LLARS_TimerOnce(section, callback, scope := "script")`

Params: section: config_timer [required]; callback: callback [required]; scope: scope = "script"

Config: timer | Return: timer_handle_or_zero | Input/block: no/no

Example: `timerID := LLARS_TimerOnce("Startup Delay", Func("DoOnce"))`

### LLARS_TimerRepeat Timing

`LLARS_TimerRepeat(section, callback, scope := "script")`

Params: section: config_timer [required]; callback: callback [required]; scope: scope = "script"

Config: timer | Return: timer_handle_or_zero | Input/block: no/no

Example: `timerID := LLARS_TimerRepeat("Script Timer", Func("Tick"))`

### LLARS_TimerStop Timing

`LLARS_TimerStop(timerRef)`

Params: timerRef: timer_handle_or_label [required]

Config: none | Return: boolean | Input/block: no/no

Example: `LLARS_TimerStop(timerID)`

### LLARS_TimerStopAll Timing

`LLARS_TimerStopAll()`

Params: (no parameters)

Config: none | Return: void | Input/block: no/no

Example: `LLARS_TimerStopAll()`

## Creator Advanced

### LLARS_ColorBuildComponentSignature Colors

`LLARS_ColorBuildComponentSignature(capture, matcher, centerX, centerY, radius := 46, seedRadius := 8, componentStep := 2, minimumSamples := 12, minimumWidth := 8, minimumHeight := 8, gridSize := 3)`

Params: capture: capture_object [required]; matcher: color_matcher [required]; centerX: integer [required]; centerY: integer [required]; radius: integer = 46; seedRadius: integer = 8; componentStep: integer = 2; minimumSamples: integer = 12; minimumWidth: integer = 8; minimumHeight: integer = 8; gridSize: integer = 3

Config: none | Return: signature_object_or_false | Input/block: no/no

Example: `signature := LLARS_ColorBuildComponentSignature(capture, matcher, centerX, centerY)`

### LLARS_ColorBuildRuneScapeComponentSignature Colors

`LLARS_ColorBuildRuneScapeComponentSignature(matcher, centerX, centerY, radius := 46, seedRadius := 8, componentStep := 2, minimumSamples := 12, minimumWidth := 8, minimumHeight := 8, gridSize := 3)`

Params: matcher: color_matcher [required]; centerX: integer [required]; centerY: integer [required]; radius: integer = 46; seedRadius: integer = 8; componentStep: integer = 2; minimumSamples: integer = 12; minimumWidth: integer = 8; minimumHeight: integer = 8; gridSize: integer = 3

Config: none | Return: signature_object_or_false | Input/block: no/no

Example: `signature := LLARS_ColorBuildRuneScapeComponentSignature(matcher, centerX, centerY)`

### LLARS_ColorBuildStableRegionSignature Colors

`LLARS_ColorBuildStableRegionSignature(frameA, frameB, left, top, right, bottom, sampleStep := 3, stabilityThreshold := 14, maximumBrightness := "", maximumChroma := "")`

Params: frameA: capture_object [required]; frameB: capture_object [required]; left: integer [required]; top: integer [required]; right: integer [required]; bottom: integer [required]; sampleStep: integer = 3; stabilityThreshold: integer = 14; maximumBrightness: number_or_blank = ""; maximumChroma: number_or_blank = ""

Config: none | Return: signature_object_or_false | Input/block: no/no

Example: `signature := LLARS_ColorBuildStableRegionSignature(frameA, frameB, left, top, right, bottom)`

### LLARS_ColorCaptureBackgroundClient Colors

`LLARS_ColorCaptureBackgroundClient(hWnd, clientWidth, clientHeight, ByRef captureMethod := "")`

Params: hWnd: hwnd [required]; clientWidth: integer [required]; clientHeight: integer [required]; captureMethod: out_string = "" [ByRef]

Config: none | Return: capture_object_or_false | Input/block: no/no

Example: `capture := LLARS_ColorCaptureBackgroundClient(hWnd, clientWidth, clientHeight, captureMethod)`

### LLARS_ColorCaptureHasContent Colors

`LLARS_ColorCaptureHasContent(bits, width, height)`

Params: bits: pixel_buffer [required]; width: integer [required]; height: integer [required]

Config: none | Return: boolean | Input/block: no/no

Example: `if LLARS_ColorCaptureHasContent(bits, width, height)`

### LLARS_ColorCaptureVisibleClient Colors

`LLARS_ColorCaptureVisibleClient(hWnd, clientWidth, clientHeight, ByRef captureMethod := "")`

Params: hWnd: hwnd [required]; clientWidth: integer [required]; clientHeight: integer [required]; captureMethod: out_string = "" [ByRef]

Config: none | Return: capture_object_or_false | Input/block: no/no

Example: `capture := LLARS_ColorCaptureVisibleClient(hWnd, clientWidth, clientHeight, captureMethod)`

### LLARS_ColorClickDenseTarget Colors

`LLARS_ColorClickDenseTarget(ByRef clientX, ByRef clientY, matcher, actionName := "Color Target", button := "left", reacquireRadius := 24, minimumReacquireDensity := 20, minimumTargetDensity := 40, targetDensityRadius := 32, targetDensityStep := 4, verifyRedClick := false, retryRedClick := false)`

Params: clientX: out_number [required] [ByRef]; clientY: out_number [required] [ByRef]; matcher: color_matcher [required]; actionName: string = "Color Target"; button: mouse_button = "left"; reacquireRadius: integer = 24; minimumReacquireDensity: integer = 20; minimumTargetDensity: integer = 40; targetDensityRadius: integer = 32; targetDensityStep: integer = 4; verifyRedClick: boolean = false; retryRedClick: boolean = false

Config: none | Return: boolean | Input/block: yes/yes

Example: `if LLARS_ColorClickDenseTarget(clientX, clientY, matcher, "Target")`

### LLARS_ColorContextMenuOverlapsTarget Colors

`LLARS_ColorContextMenuOverlapsTarget(capture, centerX, centerY)`

Params: capture: capture_object [required]; centerX: integer [required]; centerY: integer [required]

Config: none | Return: boolean | Input/block: no/no

Example: `if LLARS_ColorContextMenuOverlapsTarget(capture, centerX, centerY)`

### LLARS_ColorDensity Colors

`LLARS_ColorDensity(capture, matcher, centerX, centerY, radius := 32, sampleStep := 4)`

Params: capture: capture_object [required]; matcher: color_matcher [required]; centerX: integer [required]; centerY: integer [required]; radius: integer = 32; sampleStep: integer = 4

Config: none | Return: integer | Input/block: no/no

Example: `density := LLARS_ColorDensity(capture, matcher, centerX, centerY)`

### LLARS_ColorFindLargeClusterFromCapture Colors

`LLARS_ColorFindLargeClusterFromCapture(capture, matcher, ByRef foundX, ByRef foundY, ByRef targetScore, ByRef targetDensity, minimumClusterSamples := 260, sampleStep := 4, cellSize := 48, minimumPointDensity := 30, pointDensityRadius := 8, pointDensityStep := 2, targetDensityRadius := 32, targetDensityStep := 4, selectionMode := "spread")`

Params: capture: capture_object [required]; matcher: color_matcher [required]; foundX: out_number [required] [ByRef]; foundY: out_number [required] [ByRef]; targetScore: out_number [required] [ByRef]; targetDensity: out_number [required] [ByRef]; minimumClusterSamples: integer = 260; sampleStep: integer = 4; cellSize: integer = 48; minimumPointDensity: integer = 30; pointDensityRadius: integer = 8; pointDensityStep: integer = 2; targetDensityRadius: integer = 32; targetDensityStep: integer = 4; selectionMode: selection_mode = "spread"

Config: none | Return: boolean | Input/block: no/no

Example: `if LLARS_ColorFindLargeClusterFromCapture(capture, matcher, foundX, foundY, targetScore, targetDensity)`

### LLARS_ColorFindRuneScapeLargeCluster Colors

`LLARS_ColorFindRuneScapeLargeCluster(matcher, ByRef foundX, ByRef foundY, ByRef targetScore, ByRef targetDensity, minimumClusterSamples := 260, sampleStep := 4, cellSize := 48, minimumPointDensity := 30, pointDensityRadius := 8, pointDensityStep := 2, targetDensityRadius := 32, targetDensityStep := 4, selectionMode := "spread")`

Params: matcher: color_matcher [required]; foundX: out_number [required] [ByRef]; foundY: out_number [required] [ByRef]; targetScore: out_number [required] [ByRef]; targetDensity: out_number [required] [ByRef]; minimumClusterSamples: integer = 260; sampleStep: integer = 4; cellSize: integer = 48; minimumPointDensity: integer = 30; pointDensityRadius: integer = 8; pointDensityStep: integer = 2; targetDensityRadius: integer = 32; targetDensityStep: integer = 4; selectionMode: selection_mode = "spread"

Config: none | Return: boolean | Input/block: no/no

Example: `if LLARS_ColorFindRuneScapeLargeCluster(matcher, foundX, foundY, targetScore, targetDensity)`

### LLARS_ColorFrameMovementPercent Colors

`LLARS_ColorFrameMovementPercent(frameA, frameB, ignoreMatcher := "", step := 20, differenceThreshold := 85)`

Params: frameA: capture_object [required]; frameB: capture_object [required]; ignoreMatcher: color_matcher_or_blank = ""; step: integer = 20; differenceThreshold: integer = 85

Config: none | Return: number | Input/block: no/no

Example: `movement := LLARS_ColorFrameMovementPercent(frameA, frameB)`

### LLARS_ColorGetRuneScapeClient Colors

`LLARS_ColorGetRuneScapeClient(ByRef hWnd, ByRef clientWidth, ByRef clientHeight, requireRun := true)`

Params: hWnd: out_hwnd [required] [ByRef]; clientWidth: integer [required] [ByRef]; clientHeight: integer [required] [ByRef]; requireRun: boolean = true

Config: none | Return: boolean | Input/block: no/no

Example: `if LLARS_ColorGetRuneScapeClient(hWnd, clientWidth, clientHeight)`

### LLARS_ColorMatchComponentSignature Colors

`LLARS_ColorMatchComponentSignature(capture, signature, matcher, ByRef matchRatio := 0.0, ByRef matchedPoints := 0, searchRadius := 8, searchStep := 2, pointRadius := 2, minimumRatio := 0.45, minimumMatchedPoints := 4, minimumCoverage := 0.0)`

Params: capture: capture_object [required]; signature: signature_object [required]; matcher: color_matcher [required]; matchRatio: out_number = 0.0 [ByRef]; matchedPoints: out_integer = 0 [ByRef]; searchRadius: integer = 8; searchStep: integer = 2; pointRadius: integer = 2; minimumRatio: number_or_blank = 0.45; minimumMatchedPoints: integer = 4; minimumCoverage: number_or_blank = 0.0

Config: none | Return: boolean | Input/block: no/no

Example: `if LLARS_ColorMatchComponentSignature(capture, signature, matcher, matchRatio, matchedPoints)`

### LLARS_ColorReacquireDensePoint Colors

`LLARS_ColorReacquireDensePoint(capture, matcher, ByRef clientX, ByRef clientY, radius := 24, minimumDensity := 20, densityRadius := 6, densityStep := 2)`

Params: capture: capture_object [required]; matcher: color_matcher [required]; clientX: out_number [required] [ByRef]; clientY: out_number [required] [ByRef]; radius: integer = 24; minimumDensity: any = 20; densityRadius: any = 6; densityStep: any = 2

Config: none | Return: boolean | Input/block: no/no

Example: `if LLARS_ColorReacquireDensePoint(capture, matcher, clientX, clientY)`

### LLARS_ColorRegionSignatureDifference Colors

`LLARS_ColorRegionSignatureDifference(capture, signature, differenceThreshold := 38, ByRef changedSamples := 0, ByRef totalSamples := 0)`

Params: capture: capture_object [required]; signature: signature_object [required]; differenceThreshold: integer = 38; changedSamples: out_number = 0 [ByRef]; totalSamples: out_number = 0 [ByRef]

Config: none | Return: number_or_false | Input/block: no/no

Example: `difference := LLARS_ColorRegionSignatureDifference(capture, signature, 38, changedSamples, totalSamples)`

### LLARS_ColorResolveMatcher Colors

`LLARS_ColorResolveMatcher(matcher)`

Params: matcher: color_matcher [required]

Config: none | Return: callable_or_false | Input/block: no/no

Example: `matcherFunc := LLARS_ColorResolveMatcher(matcher)`

### LLARS_ColorSignaturePointMatches Colors

`LLARS_ColorSignaturePointMatches(capture, matcherFunc, x, y, radius := 2)`

Params: capture: capture_object [required]; matcherFunc: callback [required]; x: number [required]; y: number [required]; radius: integer = 2

Config: none | Return: boolean | Input/block: no/no

Example: `if LLARS_ColorSignaturePointMatches(capture, matcherFunc, x, y)`

### LLARS_ColorTrackConnectedComponent Colors

`LLARS_ColorTrackConnectedComponent(capture, matcher, ByRef clientX, ByRef clientY, ByRef trackingScore, radius := 118, seedRadius := 34, step := 4, minimumSamples := 78, minimumWidth := 38, minimumHeight := 38)`

Params: capture: capture_object [required]; matcher: color_matcher [required]; clientX: out_number [required] [ByRef]; clientY: out_number [required] [ByRef]; trackingScore: out_number [required] [ByRef]; radius: integer = 118; seedRadius: integer = 34; step: integer = 4; minimumSamples: integer = 78; minimumWidth: integer = 38; minimumHeight: integer = 38

Config: none | Return: boolean | Input/block: no/no

Example: `if LLARS_ColorTrackConnectedComponent(capture, matcher, clientX, clientY, trackingScore)`

### LLARS_ColorTrackRuneScapeSignature Colors

```
LLARS_ColorTrackRuneScapeSignature(signature, matcher, ByRef matchRatio, ByRef matchedPoints, trackerKey := "Color Signature", searchRadius := 8, searchStep := 2, pointRadius := 2, minimumRatio := 0.45, minimumMatchedPoints := 4, backgroundInterval := 2000, useContextMenuGuard := true, minimumCoverage := 0.0)
```

Params: signature: signature_object [required]; matcher: color_matcher [required]; matchRatio: out_number [required] [ByRef]; matchedPoints: out_integer [required] [ByRef]; trackerKey: string = "Color Signature"; searchRadius: integer = 8; searchStep: integer = 2; pointRadius: integer = 2; minimumRatio: number_or_blank = 0.45; minimumMatchedPoints: integer = 4; backgroundInterval: integer = 2000; useContextMenuGuard: boolean = true; minimumCoverage: number_or_blank = 0.0

Config: none | Return: tri_state | Input/block: no/no

Example: `state := LLARS_ColorTrackRuneScapeSignature(signature, matcher, matchRatio, matchedPoints)`

### LLARS_ColorTrackRuneScapeTarget Colors

```
LLARS_ColorTrackRuneScapeTarget(ByRef clientX, ByRef clientY, matcher, ByRef trackingScore, ByRef cameraMotion, trackerKey := "Color Target", componentRadius := 118, seedRadius := 34, componentStep := 4, minimumSamples := 78, minimumWidth := 38, minimumHeight := 38, backgroundInterval := 2000, motionThreshold := 18.0, confirmationDelay := 110, useContextMenuGuard := true)
```

Params: clientX: out_number [required] [ByRef]; clientY: out_number [required] [ByRef]; matcher: color_matcher [required]; trackingScore: out_number [required] [ByRef]; cameraMotion: out_number [required] [ByRef]; trackerKey: string = "Color Target"; componentRadius: integer = 118; seedRadius: integer = 34; componentStep: integer = 4; minimumSamples: integer = 78; minimumWidth: integer = 38; minimumHeight: integer = 38; backgroundInterval: integer = 2000; motionThreshold: number_or_blank = 18.0; confirmationDelay: integer = 110; useContextMenuGuard: boolean = true

Config: none | Return: tri_state | Input/block: no/no

Example: `state := LLARS_ColorTrackRuneScapeTarget(clientX, clientY, matcher, trackingScore, cameraMotion)`

### LLARS_PixelChangedFromRuneScapePrintWindow Conditions

```
LLARS_PixelChangedFromRuneScapePrintWindow(coordinateSection, baselineColor, ByRef currentColor, scope := "script")
```

Params: coordinateSection: config_coordinate [required]; baselineColor: color_hex [required]; currentColor: out_color_hex [required] [ByRef]; scope: scope = "script"

Config: coordinate | Return: boolean | Input/block: no/no

Example: `if LLARS_PixelChangedFromRuneScapePrintWindow("Pixel Coordinate", baselineColor, currentColor)`

### LLARS_PixelMatchesWithinTolerancePrintWindow Conditions

```
LLARS_PixelMatchesWithinTolerancePrintWindow(coordinateSection, colorSection, tolerance := 0, scope := "script")
```

Params: coordinateSection: config_coordinate [required]; colorSection: config_color [required]; tolerance: integer = 0; scope: scope = "script"

Config: coordinate,color | Return: boolean | Input/block: no/no

Example: `if LLARS_PixelMatchesWithinTolerancePrintWindow("Pixel Coordinate", "Target Color", 3)`

### LLARS_ConfigDelete Configuration

```
LLARS_ConfigDelete(section, key := "", scope := "script")
```

Params: section: config_section [required]; key: config_key = ""; scope: scope = "script"

Config: config_section | Return: boolean | Input/block: no/no

Example: `LLARS_ConfigDelete("Temporary", "value")`

### LLARS_ConfigKeyExists Configuration

```
LLARS_ConfigKeyExists(section, key, scope := "script")
```

Params: section: config_section [required]; key: config_key [required]; scope: scope = "script"

Config: config_section | Return: boolean | Input/block: no/no

Example: `if LLARS_ConfigKeyExists("Options", "mode")`

### LLARS_ConfigKeys Configuration

```
LLARS_ConfigKeys(section, scope := "script", default := "ERROR")
```

Params: section: config_section [required]; scope: scope = "script"; default: any = "ERROR"

Config: config_section | Return: newline_delimited_string_or_default | Input/block: no/no

Example: `keys := LLARS_ConfigKeys("Options")`

### LLARS_ConfigSectionExists Configuration

```
LLARS_ConfigSectionExists(section, scope := "script")
```

Params: section: config_section [required]; scope: scope = "script"

Config: config_section | Return: boolean | Input/block: no/no

Example: `if LLARS_ConfigSectionExists("Options")`

### Legacy Compatibility

#### LLARS_ConfigBool Configuration

`LLARS_ConfigBool(section, key, default := false, scope := "script")`

Params: section: config_section [required]; key: config_key [required]; default: boolean = false; scope: scope = "script"

Config: config_section | Return: boolean | Input/block: no/no

Example: `enabled := LLARS_ConfigBool("Feature", "option", false)`

#### LLARS_ConfigHasKey Configuration

`LLARS_ConfigHasKey(section, key, scope := "script")`

Params: section: config_section [required]; key: config_key [required]; scope: scope = "script"

Config: config_section | Return: boolean | Input/block: no/no

Example: `if LLARS_ConfigHasKey("Options", "mode")`

#### LLARS_ConfigHasSection Configuration

`LLARS_ConfigHasSection(section, scope := "script")`

Params: section: config_section [required]; scope: scope = "script"

Config: config_section | Return: boolean | Input/block: no/no

Example: `if LLARS_ConfigHasSection("Options")`

#### LLARS_ConfigPoint Configuration

`LLARS_ConfigPoint(section, ByRef x, ByRef y, scope := "script")`

Params: section: config_coordinate [required]; x: out_number [required] [ByRef]; y: out_number [required] [ByRef]; scope: scope = "script"

Config: coordinate | Return: boolean | Input/block: no/no

Example: `LLARS_ConfigPoint("Action Location", x, y)`

#### LLARS_ConfigRange Configuration

`LLARS_ConfigRange(section, ByRef minValue, ByRef maxValue, scope := "script")`

Params: section: config_timing_range [required]; minValue: out_number [required] [ByRef]; maxValue: out_number [required] [ByRef]; scope: scope = "script"

Config: timing_range | Return: boolean | Input/block: no/no

Example: `LLARS_ConfigRange("Action Delay", minValue, maxValue)`

#### LLARS_TimerSchedule Timing

`LLARS_TimerSchedule(labelName, section, repeat := false, scope := "script")`

Params: labelName: label_name [required]; section: config_timer [required]; repeat: boolean = false; scope: scope = "script"

Config: timer | Return: timer_handle_or_zero | Input/block: no/no

Example: `timerID := LLARS_TimerSchedule("LegacyTimer", "Script Timer", true)`

## 12. Source precedence and limitations

For an installed LLARS v2 version, project source is authoritative over this reference. In priority order: (1) installed functions and supported templates, (2) authoritative Creator API catalog and config catalog, (3) shared Config/RunCount/Selector implementations, (4) this PDF for workflow guidance. If an API is missing or renamed, stop and inspect the installed version rather than guessing.

This guide is for generating and reviewing code, not a substitute for testing AHK v1 dialogs, AHI/device settings, RuneScape client binding, or game-facing input on Windows. A generated script must be reviewed and deliberately launched before any live automation is allowed.